

Python Requests Library — Methods and Use Cases Guide

Reference guide for commonly used methods, parameters, classes, and utilities from Python's requests library.

Method / Feature	Purpose	When to Use	Example
<code>requests.get()</code>	Send an HTTP GET request	Retrieve data from APIs or webpages without modifying server state	<code>requests.get(url)</code>
<code>requests.post()</code>	Send an HTTP POST request	Submit forms, create resources, or send JSON payloads	<code>requests.post(url, json=data)</code>
<code>requests.put()</code>	Replace or update a resource	Perform full updates to existing API resources	<code>requests.put(url, json=data)</code>
<code>requests.patch()</code>	Partially update a resource	Modify only selected fields of a resource	<code>requests.patch(url, json=data)</code>
<code>requests.delete()</code>	Delete a resource	Remove API resources or records	<code>requests.delete(url)</code>
<code>requests.head()</code>	Retrieve response headers only	Check metadata or file existence without downloading content	<code>requests.head(url)</code>
<code>requests.options()</code>	Discover supported HTTP methods	Inspect API/server capabilities	<code>requests.options(url)</code>
<code>requests.request()</code>	Generic request interface	Build reusable code with dynamic HTTP methods	<code>requests.request(method, url)</code>
<code>response.json()</code>	Parse JSON response body	Handle JSON API responses	<code>response.json()</code>
<code>response.text</code>	Access response as text	Read HTML, XML, or plain text responses	<code>response.text</code>
<code>response.content</code>	Access raw binary content	Download files, PDFs, or images	<code>response.content</code>
<code>response.status_code</code>	Read HTTP status code	Validate responses and handle errors	<code>response.status_code</code>
<code>response.headers</code>	Access response headers	Inspect metadata like content type or auth	<code>response.headers</code>
<code>response.raise_for_status()</code>	Raise exception for HTTP errors	Use automatic error handling in production	<code>response.raise_for_status()</code>
<code>requests.Session()</code>	Persist cookies and settings	Reuse authentication and connections	<code>session = requests.Session()</code>
<code>session.get()</code>	GET request using a session	Maintain cookies/authentication across requests	<code>session.get(url)</code>
<code>requests.cookies</code>	Handle cookies	Work with login sessions and tracking state	<code>response.cookies</code>
<code>HTTPBasicAuth</code>	Basic authentication helper	Use APIs requiring username/password auth	<code>HTTPBasicAuth(user, pass)</code>
<code>headers</code> parameter	Send custom headers	Add tokens, user agents, or content types	<code>headers={'Authorization': 'Bearer TOKEN'}</code>
<code>params</code> parameter	Add query string parameters	Filter or search API requests	<code>params={'page': 1}</code>
<code>json</code> parameter	Send JSON request body	APIs expecting JSON payloads	<code>json={'name': 'Alice'}</code>
<code>data</code> parameter	Send form-encoded payload	HTML forms or legacy APIs	<code>data={'username': 'bob'}</code>
<code>files</code> parameter	Upload files	Send documents or images to servers	<code>files={'file': open(...)}</code>
<code>timeout</code> parameter	Set timeout duration	Prevent hanging network requests	<code>timeout=5</code>
<code>allow_redirects</code>	Control redirect behavior	Debug or block automatic redirects	<code>allow_redirects=False</code>
<code>stream=True</code>	Stream large responses	Reduce memory usage during downloads	<code>requests.get(url, stream=True)</code>

Method / Feature	Purpose	When to Use	Example
verify parameter	Enable SSL verification	Validate HTTPS certificates securely	verify=True
cert parameter	Provide SSL certificates	Use mutual TLS authentication	cert=('cert.pem', 'key.pem')
proxies parameter	Route traffic through proxies	Scraping, debugging, or network routing	proxies={'http': 'http://proxy'}
hooks parameter	Attach callback hooks	Automate logging or response processing	hooks={'response': callback}
requests.exceptions	Exception namespace	Handle network and HTTP failures	except requests.Timeout:
requests.Timeout	Timeout exception	Handle slow network conditions	except requests.Timeout:
requests.ConnectionError	Connection failure exception	Handle unreachable servers/networks	except requests.ConnectionError:
requests.HTTPError	HTTP error exception	Handle invalid HTTP responses	except requests.HTTPError:
response.iter_content()	Iterate streamed chunks	Download large files in chunks	for chunk in response.iter_content(1024):
response.iter_lines()	Iterate streamed lines	Handle logs or streaming APIs	response.iter_lines()
PreparedRequest	Prepare requests manually	Advanced request customization/debugging	req.prepare()
Session.mount()	Attach transport adapters	Configure retries or protocols	session.mount('https://', adapter)
HTTPAdapter	Retry-enabled transport adapter	Build resilient API clients	HTTPAdapter(max_retries=3)